

**BSE SENSEX PRICE DATA ANALYSIS:
FINAL PROJECT REPORT**

Nida S & Tamanna Sameer
CSIT201: Data Structures & Algorithms
Prof. Prajish Prasad
16 April 2025

Background & Data

This report is an analysis of the Sensex stock market dataset containing daily records of the Bombay Stock Exchange (BSE) Sensex index over a long time period. Each row in the dataset represents a single day of trading and includes the following columns: Date, Open, High, Low, Close.

The main objective we tried to achieve was to analyze stock behavior and patterns over time, using various aspects like trends, averages, volatility, and many more. The dataset contains 1,402 rows and 5 columns, representing the stock's Open, High, Low, Close prices, and the Date on which each trade occurred. The five key columns are:

- **Date:** The specific trading day in yyyy-mm-dd format
- **Open:** The opening price of the index or stock on that day.
- **High:** The highest price reached during the day.
- **Low:** The lowest price recorded during the day.
- **Close:** The final price at market close.

Data Processing

All information in the dataset was represented using a custom C++ struct:

```
struct SensexDay {  
    std::string date;  
    float open, high, low, close;  
    int day, month, year;  
};
```

The CSV file was read using file input operations in C++, and each row was gone through manually to extract the fields. A helper function was used to convert the date string into numerical day, month, and year components for analysis and grouping.

Sorting algorithms like merge sort and quicksort were implemented manually to order data by date, closing value, etc. Basic grouping by month or year was also done using arrays and nested loops instead of STL maps.

Analysis & Findings

Below are some of the key analyses conducted on the dataset, along with a small summary of what each analysis means and how it was implemented:

1. Daily, Monthly, and Yearly Averages

- Calculates average closing price per day, per month, and per year.
- Implemented by summing values in grouped date ranges using nested loops.
- Concepts: Array traversal, conditional grouping, date parsing.

2. Volatility Detection

- Measures how much the market fluctuates using the standard deviation of prices.
- Volatile days/months were detected by comparing ranges (High - Low).
- Concepts: Math functions, iteration, basic statistics.

3. Trend Identification (Bullish/Bearish Runs)

- Identifies the longest sequence of increasing (bullish) or decreasing (bearish) closing prices.
- Implemented with a single-pass loop checking consecutive values.
- Concepts: Longest increasing/decreasing subsequence logic.

4. Moving Averages (SMA and EMA)

- SMA: Simple moving average over a sliding window (e.g., 10 or 30 days).
- EMA: Weighted moving average that emphasizes recent data.
- Concepts: Sliding window, exponential smoothing, arithmetic.

5. Support and Resistance Level Detection

- Detects price levels where the market often reverses.
- Implemented using peak/trough detection in local neighborhoods.
- Concepts: Array scanning, pattern matching.

6. Top-N Highest and Lowest Closing Days

- Sorts days by closing value and picks the top N highest and lowest.
- Manual merge sort applied to the `close` values.
- Concepts: Sorting, array manipulation.

7. Best/Worst Months and Years

- Groups data by month/year and computes average performance.
- Compares months across years to find seasonal trends.
- Concepts: Grouping, averaging, max/min detection.

8. Sharpe Ratio / Return-Risk Metrics

- Measures average return divided by standard deviation (risk).
- Shows how consistent returns are over volatility.
- Concepts: Statistical computation, optimization.

Summary & Conclusion

This project explored historical Sensex data using a variety of data analysis techniques. By manually implementing sorting algorithms, statistical calculations, and grouping logic, we were able to understand:

- How the market performs over months and years
- Periods of high volatility or steady growth
- The most extreme bullish or bearish runs
- Identifying consistent vs. risky periods

Apart from this, the project majorly used programming concepts like data structures, file handling, sorting, algorithm design, and complexity analysis. Overall, this project shows how the real-world data can be understood using the foundational tools of data structures and algorithms.

Appendix

- Dataset Columns :
 - Date: Date of entry (dd-mm-yyyy)
 - Open: Opening Sensex value
 - High: Highest Sensex value on the day
 - Low: Lowest Sensex value on the day
 - Close: Closing Sensex value
- Computational Functions:
 - **Date parsing function to extract day/month/year** - showDataOverview(data, size);
 - **Calculate Averages** - calculateAverages(data, size);
 - **Detect Volatility** - detectVolatility(data, size, threshold);
 - **Identify Trends (Bullish or Bearish runs)** - identifyTrends(data, size)
 - **Longest Increasing/Decreasing Subsequence** - findLongestIncreasingDecreasing(data, size);
 - **SMA/EMA calculation functions** - computeSMA(data, size, windowSize);
 - **Detecting Support Resistance** - detectSupportResistance(data, size, roundFactor);
 - **Top-N Highest/Lowest Closing Days** - topNClosings(data, size, N);
 - **Best or Worst Months of the year** - bestWorstMonthsYears(data, size);

